Predicting Real vs. Fake Disaster Tweets Using Natural Language Processing

Authors: Matthew Hawkes & Benny Fishman

Team: DisasterNet Intelligence

Date: 2/12/26

Objective

The objective of this project is to build a machine learning system capable of predicting whether a tweet refers to a real disaster event or not. By accurately classifying tweets, such a system could help emergency responders, researchers, and organizations quickly identify real disaster events in real time.

The project's success criteria were:

- Achieving a strong Kaggle competition score.
- Producing a reproducible machine learning pipeline.
- Demonstrating effective preprocessing, feature engineering, and model evaluation.

Executive Summary

We developed a Natural Language Processing (NLP) pipeline that converts tweet text into numerical representations and trains classification models to predict disaster-related tweets.

Our workflow consisted of:

1. Cleaning and preprocessing the tweet text.
2. Performing exploratory data analysis to understand tweet patterns.
3. Engineering features using multiple TF-IDF vectorization methods.
4. Comparing baseline machine learning models.
5. Tuning top-performing models.
6. Generating Kaggle submissions.

After evaluating multiple models, **LinearSVC** performed best, achieving strong accuracy and F1-score performance while generalizing well to unseen data.

Problem Framing and Dataset

The dataset comes from the **NLP with Disaster Tweets** competition. Each row represents a tweet containing:

- Tweet ID
- Keyword
- Location
- Tweet text
- Target label (1 = real disaster, 0 = not a real disaster)

The challenge is that many tweets use disaster terms metaphorically, such as:

- "This exam was a disaster."
- "My schedule is a train wreck."

Therefore, models must understand context, not just keywords.

Data Preprocessing

We created two main cleaning pipelines:

clean_text

- Removes special characters and symbols.
- Tokenizes text.
- Removes stopwords.
- Applies lemmatization.

clean_text_raw

- Cleans text but **keeps stopwords**.
- Carries meaning in short tweets.

These preprocessing functions ensured consistent inputs for vectorization while preserving important linguistic signals.

Exploratory Data Analysis (EDA)

Dataset Structure

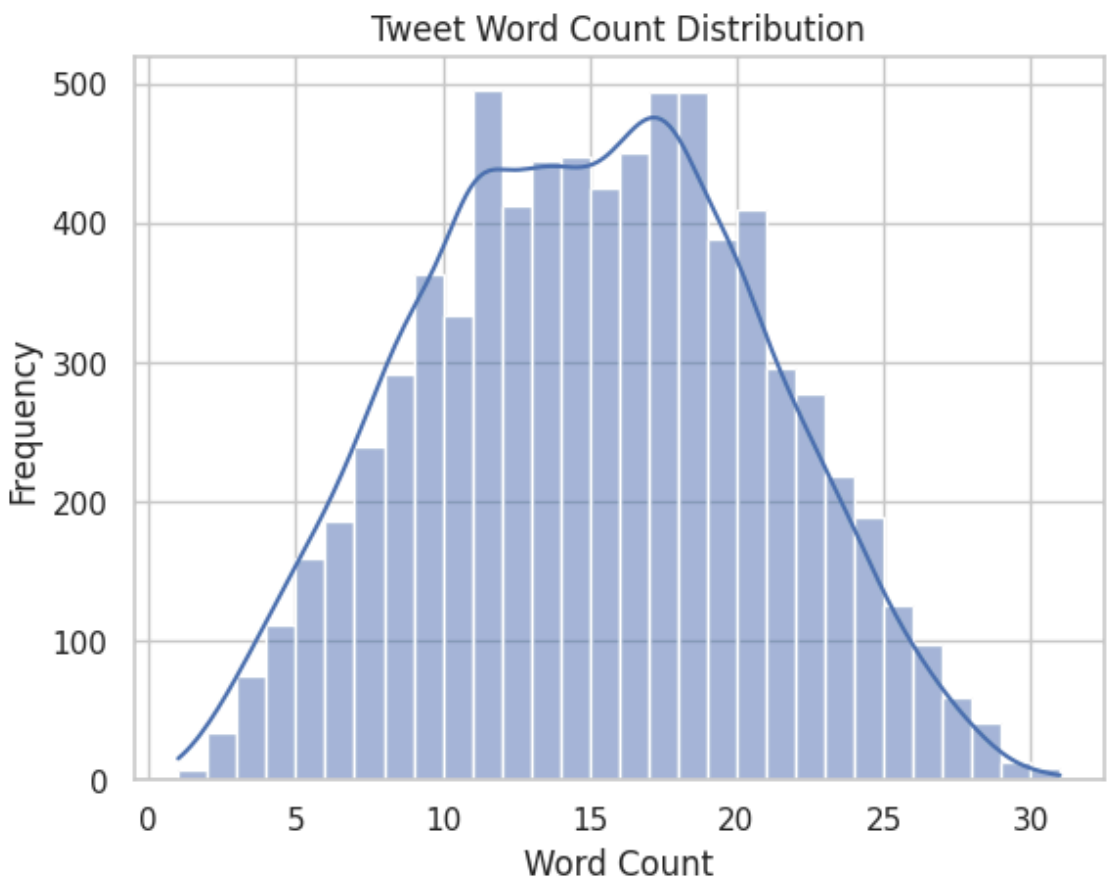
EDA revealed:

- Thousands of tweets in training data.
- Moderately balanced classes between disaster and non-disaster tweets.

```
Normalized number of tweets with target=0: 0.5710897351429701
Normalized Number of tweets with target=1: 0.42891026485702993
```

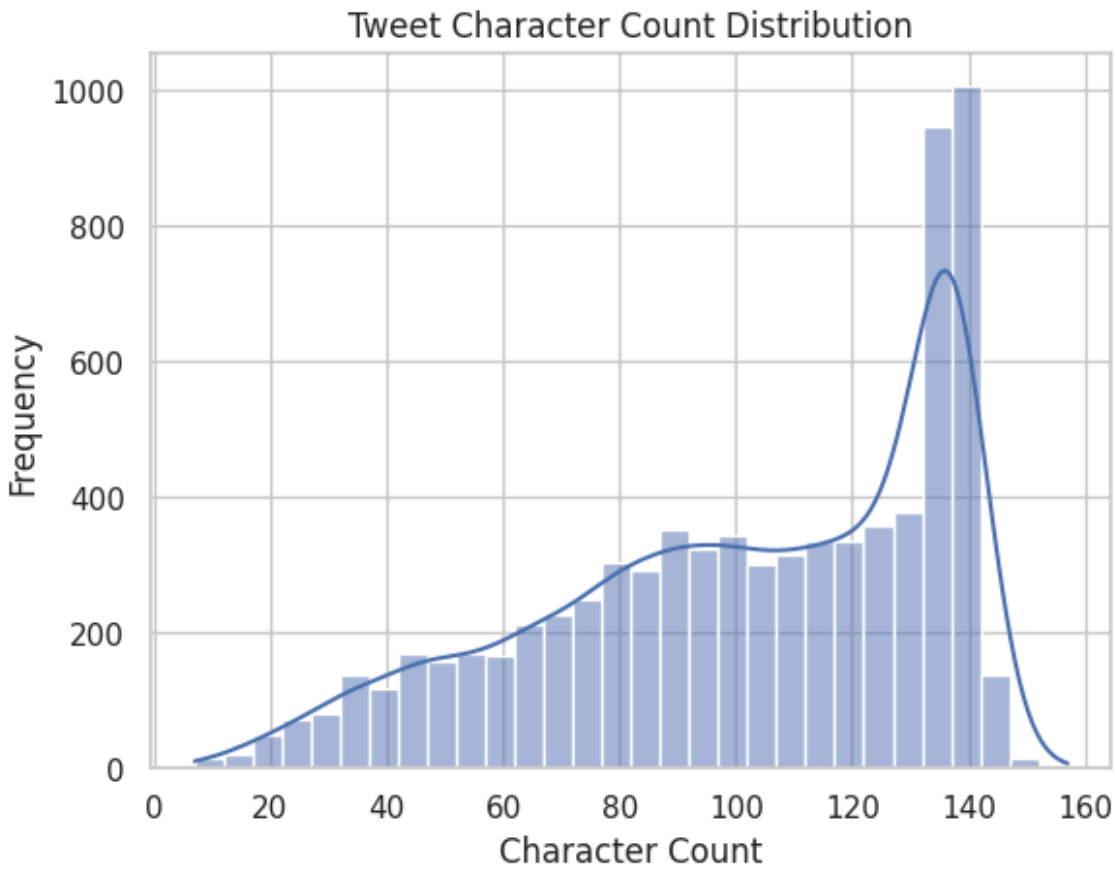
```
Number of Tweets: 7613
Column Names: id, keyword, location, text, target, tokens, cleaned_text
```

Word Count Distribution



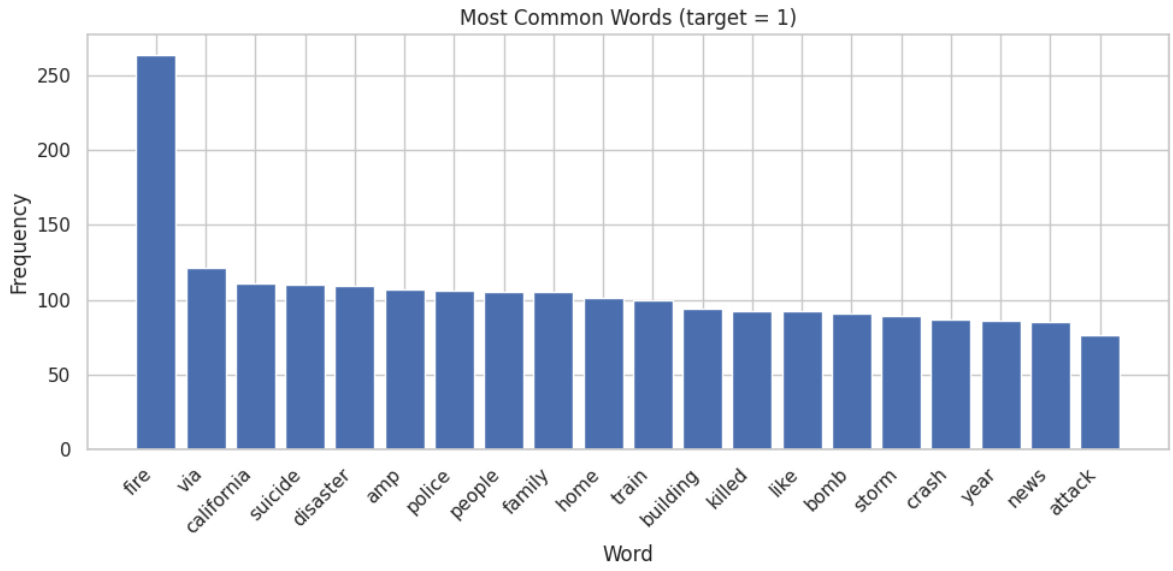
Most tweets fall within **10–20 words**, meaning tweets are short and each word carries significant importance. Since tweets are short, models must capture **meaningful patterns**, not just tweet length.

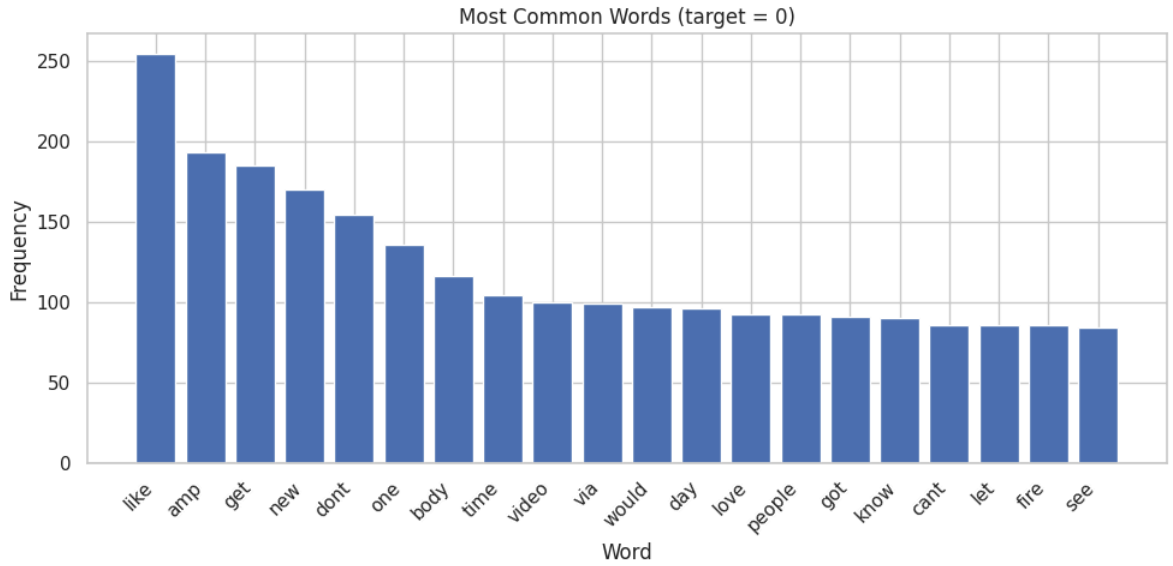
Character Count Distribution



Tweets also show limited variation in character count, confirming that classification depends more on context than size. This motivated the use of **character n-grams**, which capture subtle textual patterns.

Word Comparisons

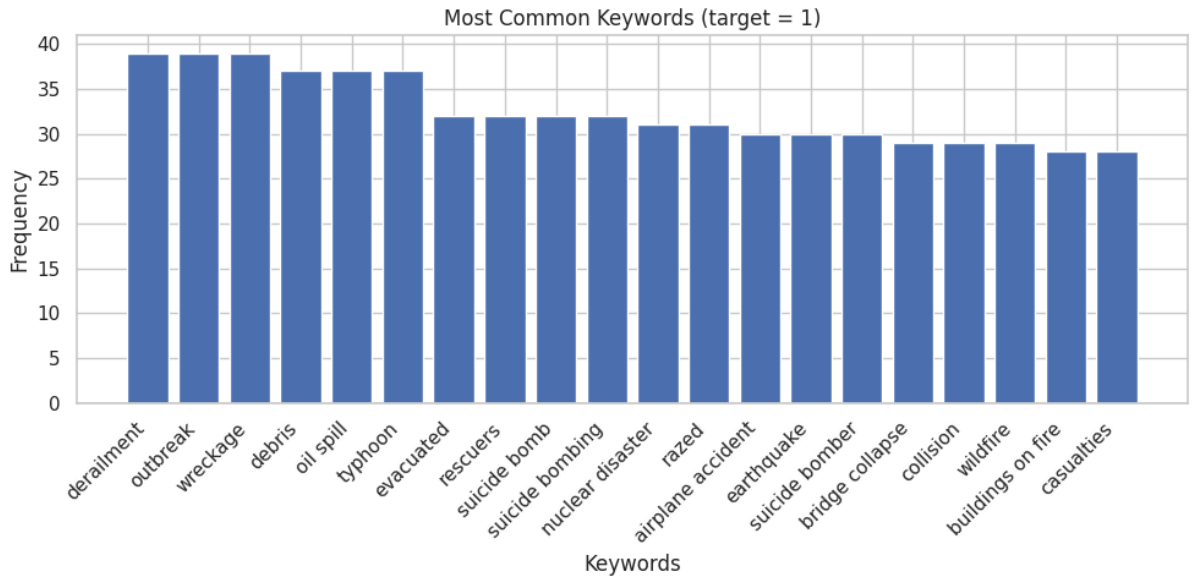


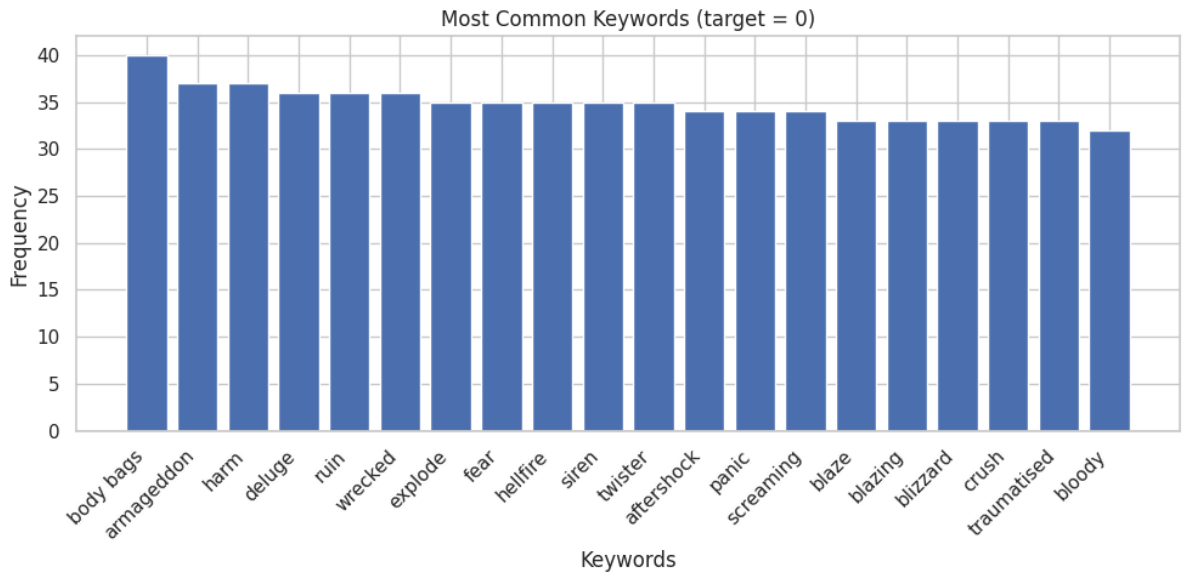


Comparing common words between classes showed:

- Certain words frequently appear in real disasters.
- Others occur more often in non-disaster tweets.
- Confirms that tweet text contains meaningful predictive patterns

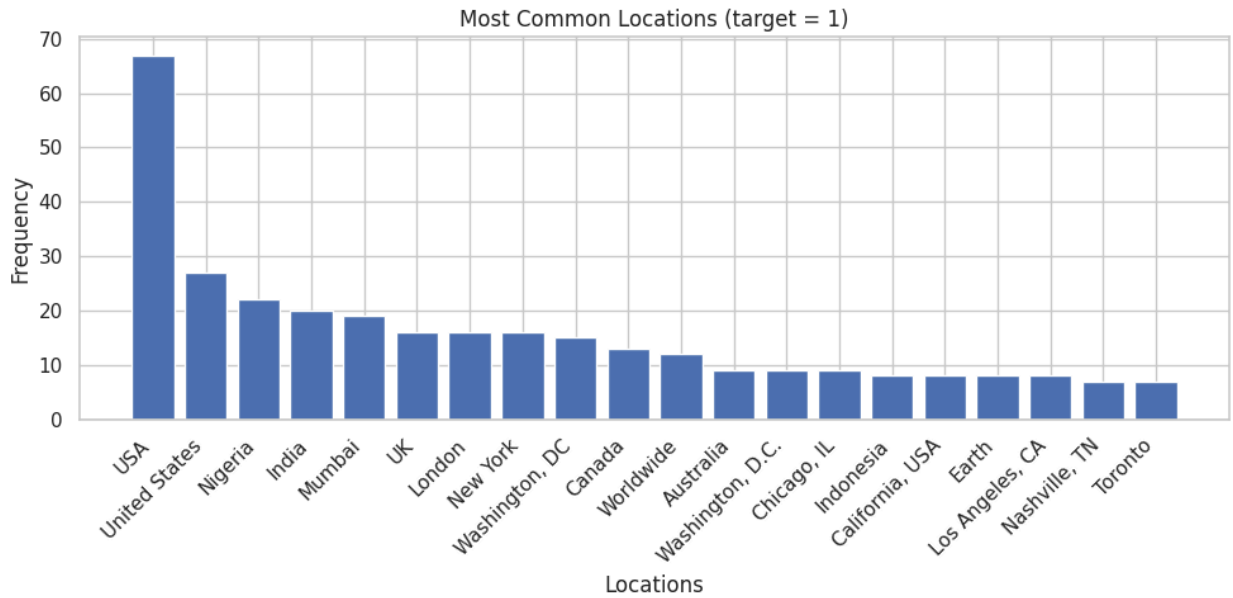
Keyword Comparisons

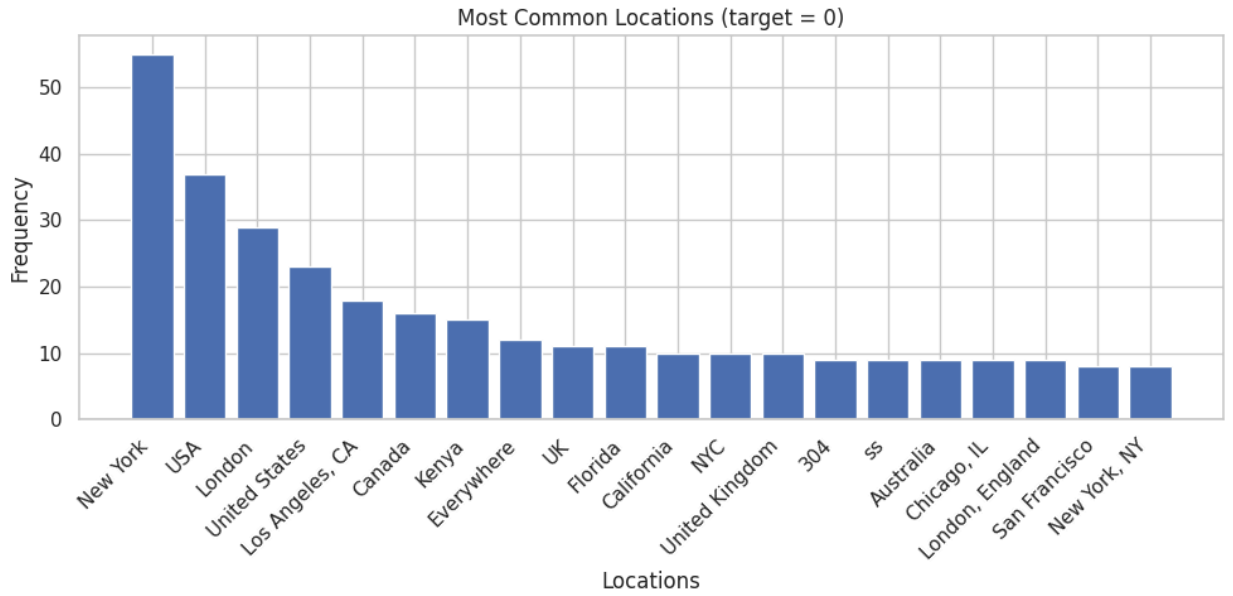




Keyword analysis also revealed the column to have a high predictive signal, which influenced our decision to vectorize it on its own.

Location Analysis





Location data proved inconsistent:

- Many tweets lacked location.
- Some contained fictional or unclear locations.

Cleaning this field would require substantial work, so it was not heavily used in modeling.

Feature Engineering: Text Vectorization

Tweets must be converted into numerical features before modeling.

We combined **four TF-IDF vectorizers** into one feature matrix:

1. Word unigrams and bigrams on lemmatized text.
2. Character n-grams (2–5 characters) on raw text.
3. Keyword unigrams.
4. Combined feature stacking into training and test matrices.

```
X_word_train  = word_tfidf.fit_transform(train_data['combined_text'])
X_char_a_train = char_tfidf_a.fit_transform(train_data['combined_raw'])
X_char_b_train = char_tfidf_b.fit_transform(train_data['combined_raw'])
X_kw_train    = kw_tfidf.fit_transform(train_data['cleaned_keyword'].fil

X_word_test   = word_tfidf.transform(test_data['combined_text'])
X_char_a_test  = char_tfidf_a.transform(test_data['combined_raw'])
X_char_b_test  = char_tfidf_b.transform(test_data['combined_raw'])
X_kw_test     = kw_tfidf.transform(test_data['cleaned_keyword'].fillna('
```

This combination allowed models to capture:

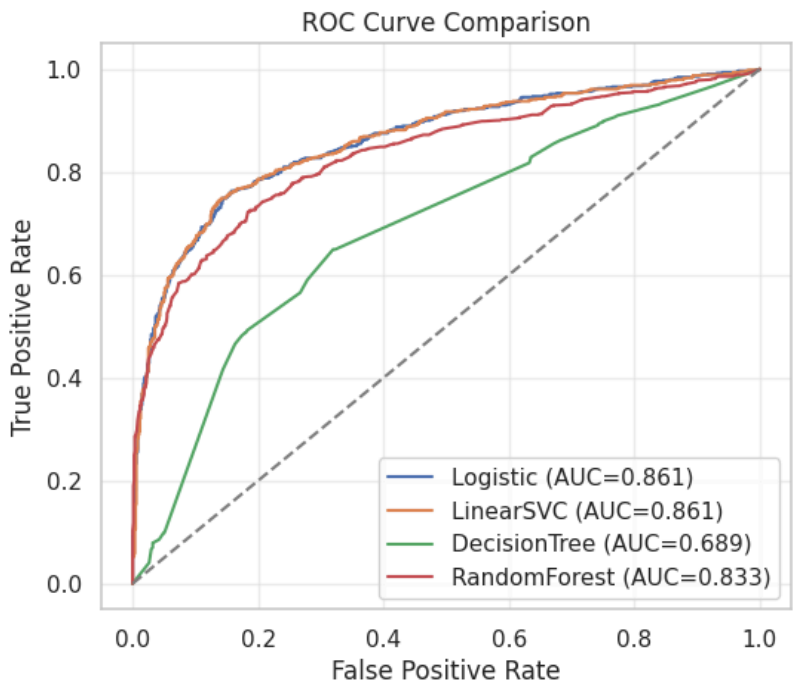
- Semantic patterns (word features)
- Subtle textual variations (character features)
- High-value keywords (kw features)

Baseline Modeling and Evaluation

We compared four baseline models:

- Logistic Regression
- LinearSVC
- Decision Tree
- Random Forest

using an 80/20 train-validation split.



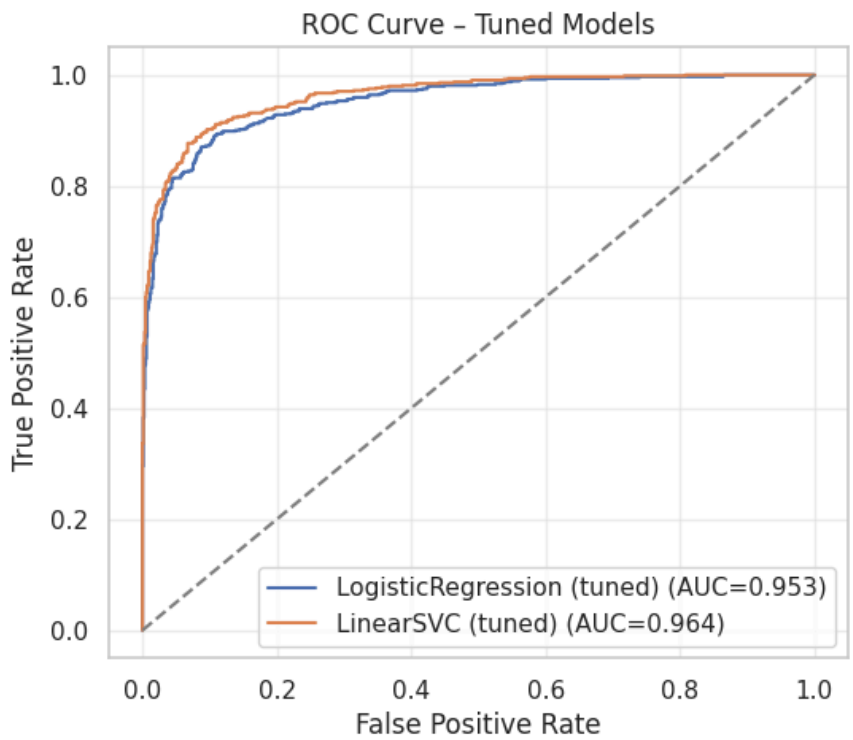
	Model	Accuracy	Precision	Recall	F1-score
1	LinearSVC	0.805665	0.774092	0.763240	0.768627
0	Logistic	0.801054	0.763158	0.767913	0.765528
3	RandomForest	0.778656	0.798828	0.637072	0.708839
2	DecisionTree	0.679842	0.663866	0.492212	0.565295

LinearSVC and Logistic Regression performed similarly in AUC, but LinearSVC achieved:

- Higher accuracy
- Better F1-score
- Stronger balance between precision and recall.

Hyperparameter Tuning

Since both baseline Logistic Regression and LinearSVC models performed well, we proceeded to tune them both.



	Model	Accuracy	Precision	Recall	F1-score
1	LinearSVC (tuned)	0.906456	0.899361	0.876947	0.888013
0	LogisticRegression (tuned)	0.892622	0.884430	0.858255	0.871146

After tuning:

- LinearSVC remained the best model.
- Performance improved slightly, validating our feature engineering.

Final Predictions and Kaggle Results

Using LinearSVC, we generated predictions for the Kaggle test set and submitted results.

Initial submission: **0.79987**
Final post-collaboration submission: **0.80202**

Collaboration Impact

Collaboration significantly improved results:

- Character + word vectorization improved signal capture.
- Extensive EDA guided modeling decisions.
- Hyperparameter tuning improved final scores.

Combining strengths led to improved performance and better project quality.

Limitations

Several challenges remain:

- Ambiguous tweets confuse models.
- Models lack deep semantic understanding.
- Dataset size limits performance.
- Location data remains difficult to utilize effectively.

Future Work

Future improvements may include:

- Deep learning models for contextual understanding.
- Incorporating tweet engagement metrics.
- Improved geographic feature engineering.
- Context-aware embeddings or transformer models.

Reflection

Key lessons learned:

- Feature engineering mattered more than complex models.
- Combining character and word TF-IDF was critical.
- EDA guided modeling decisions.
- Collaboration significantly improved outcomes.

Conclusion

This project demonstrated how NLP and classical machine learning models can effectively classify disaster-related tweets. Through careful preprocessing, feature engineering, model comparison, and tuning, we achieved strong performance and improved Kaggle results. Our

final system successfully distinguishes between real and non-disaster tweets, showing how machine learning can support disaster detection and real-time information analysis.